SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1101

COURSE OUTCOME COVERED

CO4:
implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL5)*

Assessment Tool Weightage: 30%

QUESTIONS: 6 **Total Marks:30**

Annexure A

Reverse Engineering Task

Code of Conduct:

I Mathusri P-192311363 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

---

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

---

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

---

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

---

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

---

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Q1. Legacy E-Learning Management System

1. Understanding of the Existing System

The existing E-Learning Management System contains modules for student registration, course management, content delivery, assignment evaluation, and result generation. The major problem is that user-interface code and business rules are tightly coupled. Therefore, even a small change in business logic may require modifications to several modules.

The system can be reverse engineered by identifying the major objects, their responsibilities, attributes, methods, and relationships.

2. Identification of Classes and Components

Student

- Attributes: studentId, name, email, password
- Methods: register(), login(), enrollCourse(), viewResult()

Course

- Attributes: courseId, courseName, description, duration
- Methods: createCourse(), updateCourse(), viewCourse()

Content

- Attributes: contentId, title, type, material
- Methods: uploadContent(), viewContent(), downloadContent()

Assignment

- Attributes: assignmentId, title, deadline, marks
- Methods: createAssignment(), submitAssignment(), evaluateAssignment()

Result

- Attributes: resultId, studentId, courseId, marks, grade
- Methods: calculateResult(), generateResult(), viewResult()

3. Responsibilities and Relationships

A Student can enroll in multiple Courses. A Course contains multiple Content and Assignments. An Assignment is submitted by a Student and evaluated to generate a Result.

The main relationships are:

- Student → enrolls in → Course
- Course → contains → Content
- Course → contains → Assignment
- Student → submits → Assignment
- Assignment → produces → Result

4. Proposed Layered Architecture

Presentation Layer: Provides login screens, course pages, content pages, assignment pages, and result pages.

Application/Service Layer: Coordinates use cases such as registration, enrollment, assignment submission, evaluation, and result generation.

Domain Layer: Contains Student, Course, Assignment, Content, and Result classes with their business rules.

Data Access Layer: Provides repositories or DAOs for storing and retrieving students, courses, assignments, and results.

Database Layer: Stores persistent e-learning objects and their relationships.

5. Improvement After Redesign

The redesigned system follows encapsulation, abstraction, inheritance where appropriate, and polymorphism. High cohesion ensures that each class has a focused responsibility. Low coupling ensures that changes in one layer do not unnecessarily affect other layers.
The layered structure separates presentation, business processing, domain objects, and database operations. This makes the system easier to test, maintain, extend, and scale. New features such as online examinations, certificates, discussion forums, and video learning can be added with minimal modification to existing components.

6. Conclusion

The reverse-engineered and redesigned E-Learning Management System provides better separation of concerns, high cohesion, low coupling, easier maintenance, and improved scalability. The architecture also allows new learning services to be integrated without disturbing the existing functionality.

Q2. Legacy Customer Support System

1. Analysis of Existing System

The existing Customer Support System uses multiple conditional statements to identify whether a request comes through email, chatbot, telephone, or social media. This creates tightly coupled code because every new channel requires modification of existing conditional statements.
The main limitations are:

- Excessive conditional statements
- High coupling between request processing and channels
- Difficult addition of new channels
- Poor maintainability
- Difficult testing
- Violation of the Open/Closed Principle

2. Identification of Components

SupportRequest

- Attributes: requestId, customerId, message, priority
- Methods: createRequest(), updateRequest()

SupportChannel

- Represents the common abstraction for support channels.

EmailSupport

- Method: handleRequest()

ChatbotSupport

- Method: handleRequest()

TelephoneSupport

- Method: handleRequest()

SocialMediaSupport

- Method: handleRequest()

SupportManager

- Coordinates the processing of support requests.

3. Suitable Design Pattern

The **Strategy Pattern** is suitable because each support channel can implement a common SupportChannel interface.

The system can contain:

- SupportChannel interface
- EmailSupport strategy
- ChatbotSupport strategy
- TelephoneSupport strategy
- SocialMediaSupport strategy
- SupportManager as the context

The SupportManager selects the appropriate support strategy without directly depending on the concrete implementation.

4. Refactored Processing

When a customer submits a request, the SupportManager receives the request and selects the required support channel. The selected strategy processes the request. If a new channel such as WhatsApp Support is required, a new strategy class can be added without changing the existing strategies.

5. Benefits

The redesigned system provides:

- Loose coupling
- Better extensibility
- High cohesion
- Easy testing
- Reduced conditional logic
- Better code reuse
- Easier maintenance

The design follows the Open/Closed Principle because the system can be extended with new support channels without modifying existing channel implementations.

6. Conclusion

Using behavioral design principles, particularly the Strategy Pattern, transforms the rigid legacy implementation into a flexible system. New support channels can be introduced independently, making the application scalable and maintainable.

Q3. Legacy Banking Transaction System

1. Understanding of Existing System

The legacy banking application combines account management, transaction validation, fund transfer, loan processing, transaction reporting, and database operations inside a few tightly coupled classes. This violates separation of concerns and makes testing and maintenance difficult.

2. Reverse-Engineered Domain Classes

Customer

- Attributes: customerId, name, address, contact
- Methods: updateProfile(), viewAccount()

Account

- Attributes: accountId, accountType, balance
- Methods: deposit(), withdraw(), getBalance()

Transaction

- Attributes: transactionId, amount, date, type, status
- Methods: validate(), execute(), recordTransaction()

Loan

- Attributes: loanId, amount, interestRate, duration
- Methods: calculateInterest(), approveLoan(), processLoan()

BankService

- Coordinates banking operations.

3. Proposed Layers

Presentation Layer: Provides banking screens and accepts customer requests.

Application/Service Layer: Coordinates fund transfer, loan processing, account operations, and transaction processing.

Domain Layer: Contains Customer, Account, Transaction, and Loan objects and their business rules.

Data Access Layer: Contains AccountRepository, CustomerRepository, TransactionRepository, and LoanRepository.

OODBMS: Stores persistent banking objects.

4. Suitable Design Patterns

The **Repository Pattern** can manage persistent banking objects.

The **Strategy Pattern** can be used for different loan-interest calculation policies.

The **Factory Pattern** can create different account types such as SavingsAccount and CurrentAccount.

The **Observer Pattern** can notify customers about completed transactions or suspicious activities.

5. Advantages of the Redesign

The redesigned system separates business rules from database operations. Services can be tested independently using mock repositories. Changes to the database mechanism do not require modification of business logic.

High cohesion makes each component responsible for a specific function, while low coupling allows components to change independently.

6. Conclusion

The redesigned banking system improves separation of concerns, testability, security, flexibility, and maintainability. It also provides a strong foundation for adding services such as digital wallets, investments, and online loan processing.

Q4. Legacy Hotel Reservation System

1. Analysis of Existing Architecture

The existing hotel reservation application performs OODBMS operations directly from reservation screens. Database queries for rooms, guests, bookings, and payments are repeated in multiple modules.

This creates:

- High coupling
- Code duplication
- Difficult database maintenance
- Poor separation of concerns

- Difficult testing
- Reduced reusability

2. Reverse-Engineered Components

Guest

- Attributes: guestId, name, contact, email
- Methods: registerGuest(), updateGuest()

Room

- Attributes: roomId, roomType, price, availability
- Methods: checkAvailability(), updateRoom()

Reservation

- Attributes: reservationId, guestId, roomId, checkIn, checkOut
- Methods: createReservation(), cancelReservation()

Payment

- Attributes: paymentId, amount, paymentDate, status
- Methods: processPayment(), verifyPayment()

3. Data Access Redesign

Separate data access components can be created:

- GuestDAO
- RoomDAO
- ReservationDAO
- PaymentDAO

Each DAO contains database-related operations for its corresponding object.

A Repository layer can provide higher-level operations such as:

- findAvailableRooms()
- createReservation()
- getGuestReservations()
- savePayment()

4. Layered Architecture

Presentation Layer: Handles reservation screens and user interaction.

Service Layer: Coordinates room availability, booking, cancellation, and payment.

Domain Layer: Contains Guest, Room, Reservation, and Payment objects.

Data Access Layer: Communicates with the OODBMS through DAO and Repository components.

OODBMS: Stores persistent hotel objects.

5. Benefits

The redesign eliminates repeated database code and provides a single location for persistence operations. Changes in database technology can be handled inside the data access layer without affecting the presentation or business layers.

The system achieves better modularity, reusability, maintainability, and OODBMS integration.

6. Conclusion

DAO and Repository patterns provide a clean separation between business processing and persistence management. The redesigned hotel system is easier to maintain, test, extend, and integrate with future hotel services.

Q5. Legacy Transportation Management System

1. Understanding of Existing System

The existing transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic is duplicated in several locations. Therefore, introducing a new transportation service requires changes throughout the application.

2. Reverse-Engineered Classes

Transport

- Common abstraction for all transportation types.
- Methods: start(), stop(), calculateFare()

Bus

- Attributes: busNumber, capacity, route
- Methods: start(), stop(), calculateFare()

Train

- Attributes: trainNumber, coaches, route
- Methods: start(), stop(), calculateFare()

Taxi

- Attributes: taxiNumber, driver, rate
- Methods: start(), stop(), calculateFare()

RentalVehicle

- Attributes: vehicleId, rentalRate, availability
- Methods: start(), stop(), calculateFare()

3. Object Relationships

Bus, Train, Taxi, and RentalVehicle implement or inherit from the common Transport abstraction. This allows the application to work with all transportation objects through a common interface.

4. Factory Method Design

A **Factory Method Pattern** can be introduced to centralize object creation.

A TransportFactory can provide methods for creating:

- Bus objects
- Train objects
- Taxi objects
- RentalVehicle objects

The application requests a transport object from the factory rather than directly creating concrete objects throughout the system.

5. Improved Design

If a new transportation type such as ElectricVehicle is introduced, a new class and corresponding factory implementation can be added. Existing application logic does not need major changes.

Polymorphism allows all transportation objects to be handled through the Transport abstraction.

6. Benefits

The redesigned system provides:

- Reduced code duplication
- Centralized object creation
- Loose coupling

- Better code reuse
- Improved scalability
- Easy addition of new transportation types
- Easier testing and maintenance

7. Conclusion

The Factory Method Pattern provides a flexible mechanism for creating transportation objects. Combined with abstraction and polymorphism, it makes the system extensible and reduces the impact of future changes.

Q6. Legacy Healthcare Monitoring System

1. Understanding of Existing System

The healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete monitoring devices and storage components. This results in low cohesion and high coupling.

2. SOLID Violations

Single Responsibility Principle (SRP):

A single class performs patient management, sensor processing, alert generation, database storage, and report generation. These responsibilities should be separated.

Open/Closed Principle (OCP):

Adding a new monitoring device may require modification of existing classes.

Dependency Inversion Principle (DIP):

The system directly depends on concrete sensors and storage implementations instead of abstractions.

3. Reverse-Engineered Components

Patient

- Attributes: patientId, name, age, contact
- Methods: updateDetails(), viewRecord()

Sensor

- Common interface for monitoring devices.
- Method: readData()

HeartRateSensor

- Implements Sensor.

TemperatureSensor

- Implements Sensor.

BloodPressureSensor

- Implements Sensor.

AlertService

- Generates alerts when readings exceed defined limits.

MedicalRecordRepository

- Stores and retrieves medical records.

ReportService

- Generates patient monitoring reports.

4. Redesigned Architecture

Presentation Layer: Displays patient information, sensor readings, alerts, and reports.
Application Layer: Coordinates monitoring operations.
Domain Layer: Contains Patient, SensorReading, MedicalRecord, and healthcare rules.
Infrastructure/Data Access Layer: Provides sensor communication, repository implementations, and database services.

5. Suitable Design Patterns

The **Strategy Pattern** can support different monitoring or analysis algorithms.
The **Observer Pattern** can notify AlertService when abnormal sensor readings are detected.
The **Factory Pattern** can create different sensor objects.
The **Repository Pattern** can manage persistent medical records.

6. Benefits of the Redesigned System

The redesigned system achieves high cohesion because each class has a focused responsibility.
Loose coupling is achieved through interfaces such as Sensor and MedicalRecordRepository.
A new sensor can be added without changing the monitoring service. Different repository implementations can also be used for testing or production environments.
Dependency Injection can provide required sensor and repository implementations to services, improving testability.

7. Overall Result

The redesigned healthcare system becomes:

- More flexible
- Easier to test
- Easier to maintain
- More reusable
- More scalable
- Less dependent on concrete implementations

8. Conclusion

Applying SRP, OCP, and DIP together with suitable design patterns transforms the tightly coupled healthcare application into a modular object-oriented system. The redesigned architecture supports future sensors, monitoring algorithms, alert mechanisms, and storage technologies without major changes to existing business logic.